

研讨题 5-1

条件码与条件分支

条件码

条件码在运算后发生改变

ZF (Zero Flag): 当该次运算结果为 0 时被置为 1, 否则置为 0。

SF (Sign Flag): 当运算结果的符号位 (最高位) 为 1 时被置为 1, 否则置为 0。

CF (Carry Flag): 当两个 unsigned 类型的数作运算因 进位/借位 而发生溢出时置为 1, 否则置为 0。

OF (Overflow Flag): 当两个 signed 类型的数做运算而发生符号位溢出时置为 1, 否则置为 0。

注意进位和溢出相对独立, 并不一定同时发生或不发生

example : addq指令 $t = a + b$

$ZF = (t == 0)$ $SF = (t < 0)$ $CF = a + b >= (2^{64})$ $OF = (a > 0 \ \&\& \ b > 0 \ \&\& \ t < 0) \ || \ (a < 0 \ \&\& \ b < 0 \ \&\& \ t >= 0)$

一些指令对条件码的影响

- 算数操作：
 - add sub 设置全部条件码
 - inc dec 设置OF,ZF 不设置CF (表示循环迭代, 不干扰多精度运算进位链)
- 逻辑运算：
 - and, or ,xor, 影响ZF SF 清零 CF 和 OF
 - not 无影响 (不涉及比较, 算数)
- 位操作: shl, shr ... CF设置为最后一个被移出的位 (保存信息), OF清零
- 注意 lea 操作对条件码无影响

条件码指令

- ☐ cmpq s1, s2 计算 $s2 - s1$
对比sub指令 ALU操作相同, 都会修改条件码, 但cmp 不会保存结果, 只用于条件码设置
- ☐ testq s1, s2 计算 $s1 \& s2$
典型用法:
 - testq %rax %rax 计算结果表示%rax值本身 通过ZF SF判断正负
 - testq 其中一个操作数为mask
 - 例如测试奇数 testq \$1, %rax

☐ setX

setX	操作	描述
sete D	$D \leftarrow ZF$	等于 0 时设置
setne D	$D \leftarrow \sim ZF$	不等于 0 时设置
sets D	$D \leftarrow SF$	负数时设置
setns D	$D \leftarrow \sim SF$	非负数时设置
setg D	$D \leftarrow \sim(SF \wedge OF) \& \sim ZF$	大于 (有符号)
setge D	$D \leftarrow \sim(SF \wedge OF)$	大于或等于 (有符号)
setl D	$D \leftarrow (SF \wedge OF)$	小于 (有符号)
setle D	$D \leftarrow (SF \wedge OF) \vee ZF$	小于或等于 (有符号)
seta D	$D \leftarrow \sim CF \& \sim ZF$	大于 (无符号)
setb D	$D \leftarrow CF$	小于 (无符号)

跳转指令

直接跳转 vs 间接跳转

jmp Label jmp *operand

PC相对： 将目标指令地址与跳转指令下一条指令差作为编码 （位置无关性）

条件跳转

jX	条件	jX	Condition
jmp	1	jg	$\sim(SF \wedge OF) \& \sim ZF$
je	ZF	jge	$\sim(SF \wedge OF)$
jne	$\sim ZF$	jl	$(SF \wedge OF)$
js	SF	jle	$(SF \wedge OF) \vee ZF$
jns	$\sim SF$	ja	$\sim CF \& \sim ZF$
		jb	CF

条件传送

cmovX S R 当条件码X满足时 S->R

几个研讨题

		-----	-----	-----	-----	-----	-----
		below (unsigned)					
		%rax				SF	CF
						OF	ZF
xorq	%rax, %rax	0x0000	0000	0000	0000	0	0
subq	\$1, %rax	0xFFFF	FFFF	FFFF	FFFF	1	1
cmpq	\$2, %rax	0xFFFF	FFFF	FFFF	FFFF	0	0
setl	%al	0xFFFF	FFFF	FFFF	FF01	1	0
movzblq	%al, %eax	0x0000	0000	0000	0001	0	0

Note: **setl** and **movzblq** do not modify condition codes

35

拓展研讨：使用条件传送指令的汇编代码，把两种结果都计算出来了，做的工作更多，但是相比之前的条件转移代码实现形式，为什么性能反而会更高？

核心：控制依赖转换为数据依赖

跳转时进行分支预测，若预测失败pipeline中断，造成很多个时钟周期损失，实际期望时间更长

因此简单计算时用条件传送指令更好

什么时候不能用条件传送？

- `val = Test(x) ? Hard1(x) : Hard2(x);` hard1 hard2 复杂
- `val = p ? *p : 0;` 危险计算
- `val = x > 0 ? x*=7 : x+=3;` 有副作用的计算